System Design Q&A

Q1. What is scalability?

Scalability is the ability of a system to **handle increasing users, requests, or data without a major drop in performance**.
Suppose your application currently handles 1,000 users. Later, 1 lakh users start using it. If you can increase the system's capacity to handle them properly, your system is scalable.

Q2. What is vertical scaling vs horizontal scaling?

Vertical scaling means increasing the power of an existing server.
Horizontal scaling means adding more servers.

Vertical:

1 Server
↓
Increase CPU/RAM
↓
More powerful server

Horizontal:

Server 1
Server 2
Server 3
↓
Handle requests together

Easy way to remember:

Vertical → make one machine stronger
Horizontal → add more machines

Q3. What is load balancing?

A **load balancer distributes incoming requests across multiple servers** so that one server doesn't receive all the traffic.

Load Balancer
/ | \
↓ ↓ ↓
Server 1 Server 2 Server 3

Suppose 3,000 users send requests. The load balancer can distribute those requests across the available servers.

Instead of 1 cashier handling 100 customers, a store opens 3 counters and distributes customers among them.

Q4. What is caching?

Caching stores frequently accessed data in a faster location so that future requests can be served more quickly.
Instead of repeatedly fetching the same data from a database, we can store it in a cache.

Without cache:

User
↓
Application
↓
Database
↓
Response

With cache:

User
↓
Application
↓
Cache → Data found
↓
Response

If the data is not in the cache, the application can fetch it from the database and potentially store it in the cache.

If you frequently use the same textbook, keeping it on your desk is faster than going to the library every time.

Q5. What is a CDN?

CDN (Content Delivery Network) is a distributed network of servers that delivers content from locations closer to users.

It is commonly used for static content such as:

Images
CSS
JavaScript
Videos

Fonts

Suppose your main server is in India but a user is in the US.

Without CDN:

US User → India Server

With CDN:

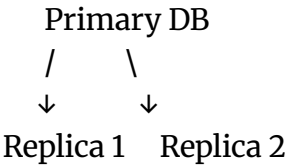
US User → Nearby CDN Server
The nearby CDN server can serve cached static content, reducing latency.

Q6. What is a database replica?

A **database replica is a copy of a database that is kept synchronized with another database**.

Replicas can be used for:

- Handling more read traffic
- Improving availability
- Disaster recovery



Write operations may go to the primary database, while some read operations can be served by replicas, depending on the architecture.

Imagine having multiple photocopies of the same document so several people can read it simultaneously.

Q7. What is the difference between SQL and NoSQL?

SQL databases generally organize data using structured tables and relationships and commonly use SQL for querying.
NoSQL is a broad category of databases that use models such as document, key-value, wide-column, or graph databases.

SQL:

Users

id | name | age
1 | Rahul | 22

2 | Anu | 23

A document-oriented NoSQL database might store:

```
{
 "id": 1,
 "name": "Rahul",
 "age": 22
}
```

Simple interview answer:

> "SQL is commonly preferred when we need structured relational data and strong relational querying. NoSQL can be useful when the data model or scaling requirements fit document, key-value, graph, or other non-relational models."

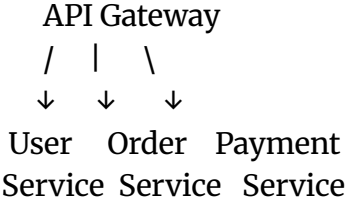
Important:
Don't say **"SQL is not scalable and NoSQL is scalable."** Both SQL and NoSQL systems can scale; the appropriate choice depends on the workload and requirements.

Q8. What is an API Gateway?

An **API Gateway is a component that acts as an entry point between clients and backend services**.

It can handle things such as:

- Routing
- Authentication
- Rate limiting
- Request processing
- Logging
- Sometimes response aggregation



Instead of the mobile application directly communicating with every backend service, it can communicate through the API Gateway.

Q9. What is a message queue?

A **message queue allows one component to send messages that another component can process asynchronously**.

The sender doesn't always have to wait for the receiver to finish processing.

Order Service
↓
Message Queue
↓
Email Service

When a customer places an order, the order service can put:

"Send order confirmation email"

into the queue.

The email service processes it later.

A restaurant takes your order and puts it in a queue for the kitchen. The waiter doesn't need to stand in the kitchen until the food is prepared.

Q10. What is a microservice?

A **microservice is a small, independently deployable service designed around a specific business capability**.

A large application can be divided into multiple services.

An e-commerce application could have:

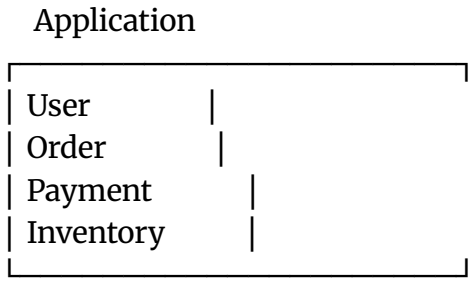
User Service
↓
Order Service
↓
Payment Service
↓
Inventory Service
Each service can be developed and deployed independently, while communicating through APIs or messaging.

Q11. What is the difference between a Monolith and Microservices?

Answer:

A **monolithic application** is typically built and deployed as one application.
A **microservices architecture** splits the application into multiple independently deployable services.

Monolith:



Microservices:

User Service
↕
Order Service
↕
Payment Service
↕
Inventory Service

In a monolith, changing the payment module may require deploying the whole application.

With microservices, the payment service can potentially be deployed independently.

Important interview point:

Don't say:

> "Microservices are always better."

They introduce additional complexity such as:

- Network communication
- Distributed debugging
- Deployment complexity
- Service discovery
- Data consistency challenges

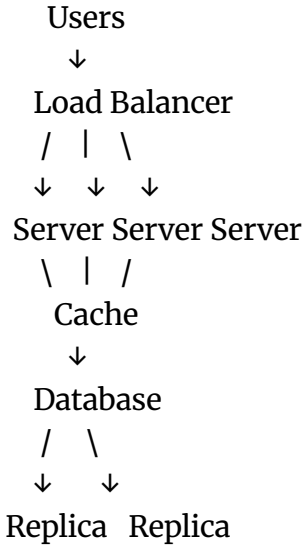
For a small application, a monolith can be the better choice.

Q12. What happens when 1 million users access your application?

I would first identify the bottleneck and then scale the system.

I would first identify the bottleneck and then scale the system.

A simple architecture could look like:

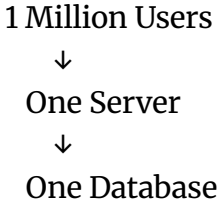


We can improve the system using:

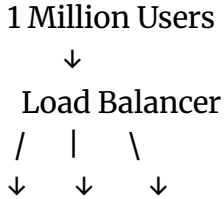
1. **Load balancing** → distribute requests across servers.
2. **Horizontal scaling** → add more application servers.
3. **Caching** → reduce repeated database queries.
4. **Database replicas** → distribute read traffic.
5. **CDN** → serve static content closer to users.
6. **Message queues** → move heavy/asynchronous work away from request processing.
7. **Database optimization** → indexes, query optimization, partitioning/sharding when appropriate.
8. **Monitoring** → identify CPU, memory, latency, errors, database load, and other bottlenecks.

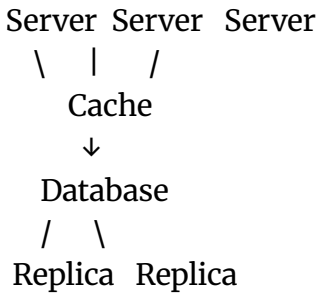
Suppose 1 million users open a news application at the same time.

Instead of:



which can become a bottleneck, we could have:





Static images, CSS, and JavaScript can additionally be served through a CDN.

Interview-ready answer:

> "If 1 million users access the application, I wouldn't immediately assume I need a particular technology. First I'd identify the bottleneck. I could horizontally scale application servers behind a load balancer, use caching to reduce database load, use database replicas for read-heavy workloads, use a CDN for static content, and use message queues for asynchronous operations. I'd also monitor latency, errors, CPU, memory, and database performance to determine where further scaling is needed."

That last answer is the one I'd **practice saying out loud**, because system-design interviews are less about memorizing definitions and more about showing that you can reason about **where the bottleneck is and how to remove it**.